PROJECT REPORT

On

AI-Based Code Plagiarism Detection System

Submitted
Of
"TAE-1(Project Based Learning)"
Under the subject
Artificial Intelligence

(VI Semester, B.Tech CSE)
(ACADEMIC SESSION 2025-26)

Criteria	Rubrics	Marks
AI Technique(s) (2M)	• Selection of suitable technique	
Problem Representation (4M)	• Representation of the method and knowledge required by the statement	
Tools explored (2M)	• Use of AI tool from industry perspective	
Outcome (2M	• Analysis of project working	
Total Marks		

Submitted By:

Name: Sonal Shukla

USN: CS23176

S. B. JAIN INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NAGPUR

(AN AUTONOMOUS INSTITUTION AFFILIATED TO RASHTRASANT TUKADOJI MAHARAJ NAGPUR UNIVERSITY, NAAC ACCREDITED WITH 'A' GRADE)

1. Title: AI-Based Code Plagiarism Detection System using Similarity Analysis and Visualization

2. Problem Statement:

With the increasing use of online coding platforms, academic submissions, and open source repositories, code plagiarism has become a major issue. Students and developers often copy code with minor modifications, making it difficult to detect using traditional methods.

Existing systems mainly provide a similarity percentage but lack:

- Detailed analysis of copied sections
- Visualization of similarities
- Suggestions to reduce plagiarism

Hence, there is a need for an intelligent system that not only detects plagiarism but also explains and visualizes it.

3. Objective:

With the increasing use of online coding platforms, academic submissions, and open source repositories, code plagiarism has become a major issue. Students and developers often copy code with minor modifications, making it difficult to detect using traditional methods.

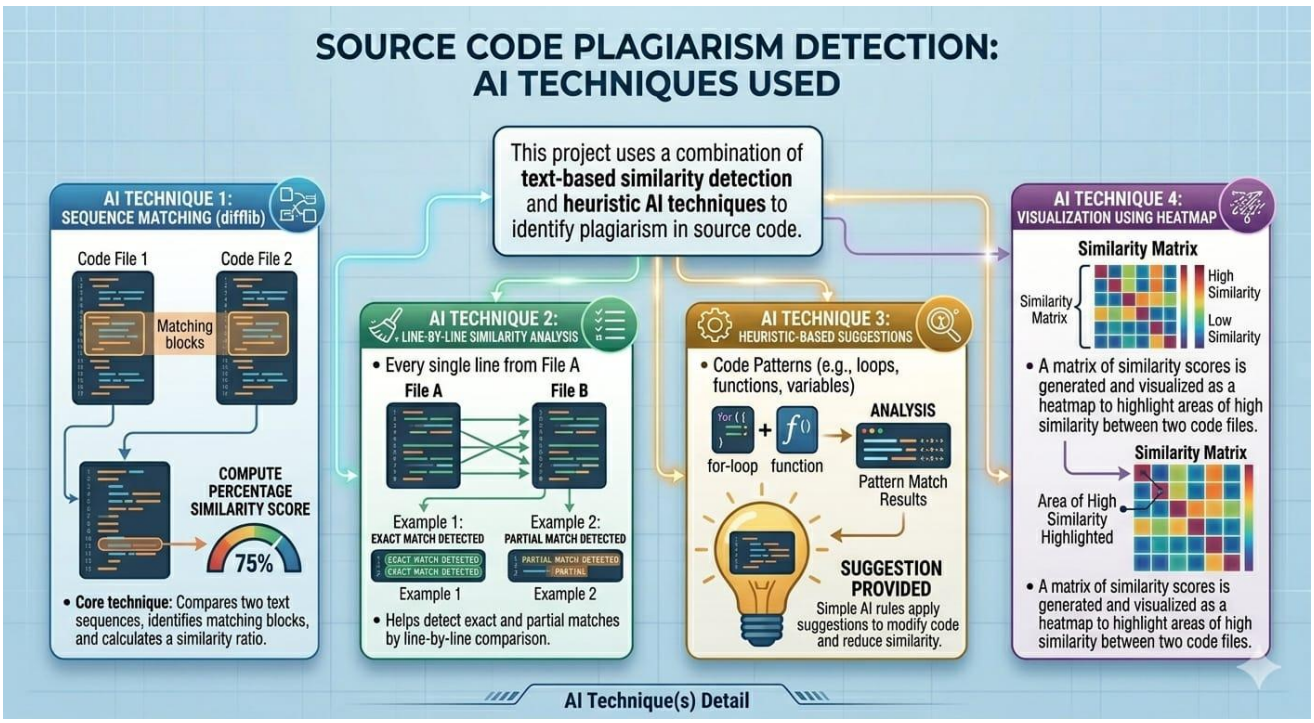
Existing systems mainly provide a similarity percentage but lack:

- Detailed analysis of copied sections
- Visualization of similarities
- Suggestions to reduce plagiarism

Hence, there is a need for an intelligent system that not only detects plagiarism but also explains and visualizes it.

4. Detail Document

AI Technique(s) Used



This project uses a combination of text-based similarity detection and heuristic AI techniques to identify plagiarism in source code.

- Sequence Matching Algorithm (difflib):

  The core technique used is the *SequenceMatcher* algorithm, which compares two sequences of text and calculates a similarity ratio. It identifies matching blocks between two code files and computes a percentage similarity score.

- Line-by-Line Similarity Analysis:

  Each line of code from one file is compared with every line of the second file. This helps in detecting exact and partial matches.

- Heuristic-Based Suggestions:

  Simple AI rules are applied to analyze patterns in code (such as loops, functions, variables) and suggest modifications to reduce similarity.

- Visualization using Heatmap:

  A matrix of similarity scores is generated and visualized as a heatmap to highlight areas of high similarity between two code files.

Problem Representation

The problem of plagiarism detection is represented as a text similarity and pattern matching problem.
Input Representation:

- Two source code files (Python/Java/C++)
- Code is treated as raw text input

Preprocessing:

- Removal of comments (to avoid false similarity)
- Removal of extra spaces and blank lines
- Conversion into structured format (list of lines)

Modeling Approach:

- Code is represented as a sequence of lines
- Each line acts as a unit for comparison
- A similarity matrix is created where:
  - Rows represent lines from Code 1
  - Columns represent lines from Code 2
  - Each cell contains similarity score between two lines

Knowledge Representation:

- Matching patterns between lines
- Structural similarity (loops, functions, variables)
- Identical or near-identical statements

Processing Steps:

1. Input code files are uploaded
2. Code is preprocessed and cleaned
3. Similarity score is calculated
4. Matching lines are identified
5. Heatmap is generated for visualization
6. Suggestions are provided
7. Modified code is generated

Tools & Technologies Used

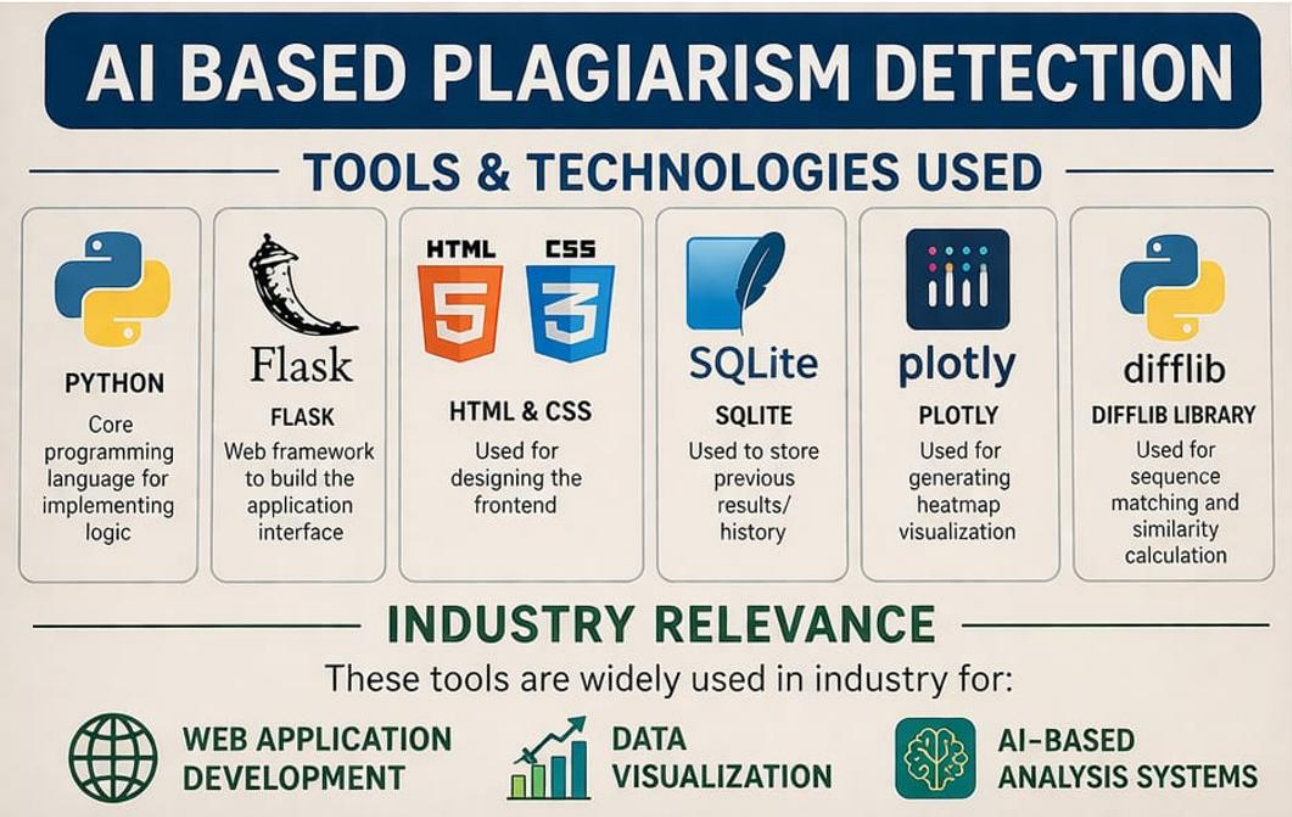
The following tools and technologies are used in this project:

- Python: Core programming language for implementing logic
- Flask: Web framework to build the application interface
- HTML & CSS: Used for designing the frontend
- SQLite: Used to store previous results/history
- Plotly: Used for generating heatmap visualization
- difflib Library: Used for sequence matching and similarity calculation

Industry Relevance:

These tools are widely used in industry for:

- Web application development
- Data visualization
- AI-based analysis systems



5. Proposed Solution

The proposed system is an AI-based Code Plagiarism Detection System that compares two code files and provides detailed analysis along with visualization.

Working of the System:

1. Input Module:
   - User uploads two code files (Python/Java/C++) through the web interface.
2. Preprocessing Module:
   - Removes comments and unnecessary spaces.
   - Converts code into a structured format (line-by-line representation).
3. Similarity Detection Module:
   - Uses *SequenceMatcher algorithm* to calculate similarity between the two codes.
   - Generates an overall similarity percentage.
4. Line Matching Module:
   - Compares each line of code with another file.
   - Identifies copied or highly similar lines.
5. Visualization Module:
   - Creates a heatmap representing similarity between lines.
   - Darker regions indicate higher similarity.

6. Suggestion Module:

• Provides recommendations like:

- Renaming variables
- Changing loops or functions
- Modifying structure

7. Auto-Modification Module:

• Generates a plagiarism-free version of code by:

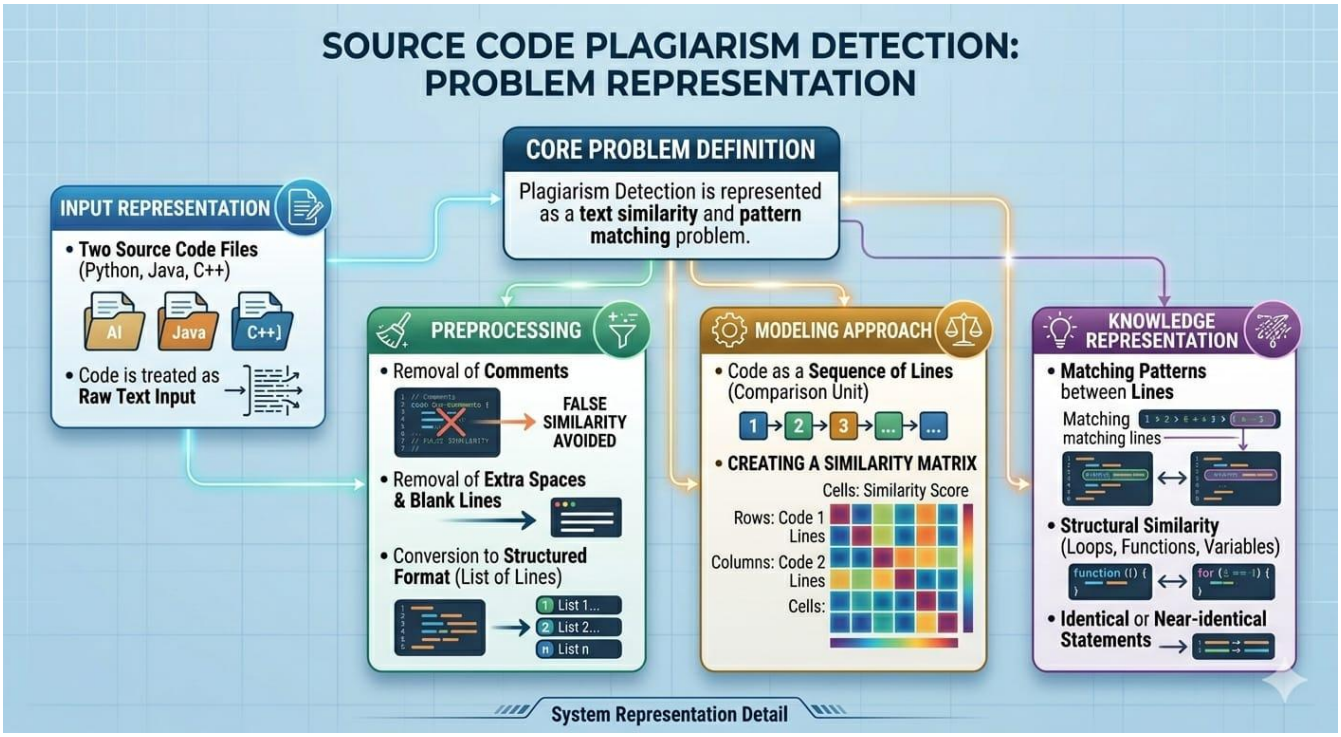
- Renaming variables
- Slight structural changes

8. Database Module:

• Stores previous results including:

- Language
- Similarity score
- Result status

System Architecture (Flow):



Input → Preprocessing → Similarity Calculation → Line Matching → Heatmap Visualization → Suggestions → Auto Code Generation → Result Storage

Advantages of Proposed System:

• Provides both numerical and visual analysis

• Easy to use web interface

• Helps users understand and reduce plagiarism

• Maintains history for future reference

6. Result/Outcome

The developed system successfully achieves the objectives of plagiarism detection and analysis.

Key Results:

- The system calculates accurate similarity scores between two code files
- It identifies and highlights copied lines clearly
- Generates useful suggestions to reduce plagiarism
- Produces a modified (plagiarism-free) version of code
- Displays a heatmap visualization for better understanding of similarity patterns
- Stores results for future reference

Performance Analysis:

- Works efficiently for small to medium-sized programs
- Provides quick results with minimal processing time
- Improves understanding of plagiarism through visualization

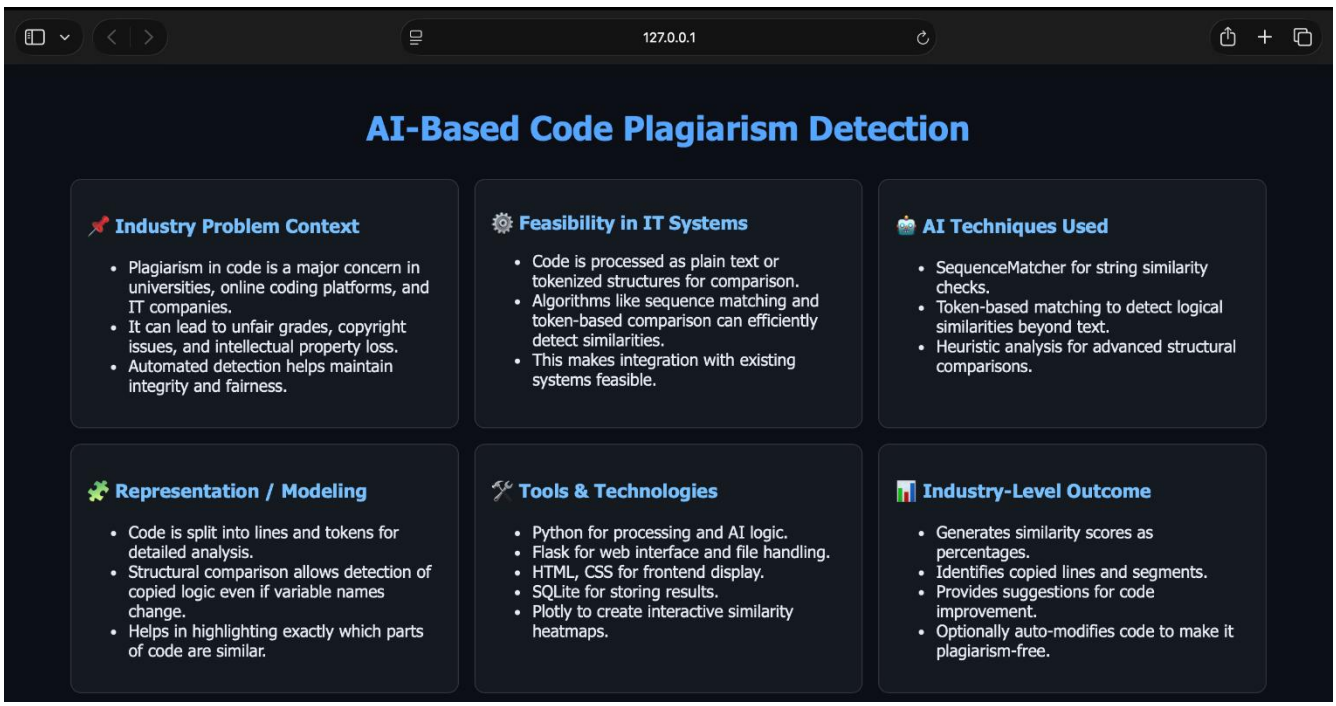
Conclusion of Results:

The system proves to be effective in detecting and analyzing code plagiarism by combining:

- Similarity detection
- Visualization
- Automated suggestions

It not only detects plagiarism but also helps users improve code originality, making it useful in academic and industry environments.

7. Screen Shots



🌍 Real-World Applications

- Universities and colleges for evaluating student submissions.
- Coding platforms like LeetCode, HackerRank to prevent cheating.
- Companies during recruitment or code audits.
- Open-source projects for checking license compliance.

Select Language

Python

Upload File 1

Choose File no file selected

Upload File 2

Choose File no file selected

Check Plagiarism

Previous Results

ID	Language	Score	Result
1	python	83.07	⚠️ Possible Plagiarism Detected

file1.py

Users > laxmikantrakhade > Desktop > sonal ai > file1.py > ...

```python
# Program to print numbers and calculate sum

def calculate_sum(n):
    total = 0
    for i in range(n):
        total += i
    return total


for i in range(5):
    print(i)

result = calculate_sum(5)
print("Sum is:", result)
```

file2.py

Users > laxmikantrakhade > Desktop > sonal ai > file2.py > ...

```python
# Program to print numbers and compute sum

def compute_sum(n):
    total = 0
    for num in range(n):
        total = total + num
    return total


for x in range(5):
    print(x)

output = compute_sum(5)
print("Sum is:", output)
```

Similarity Score: 83.07%

⚠️ Possible Plagiarism Detected

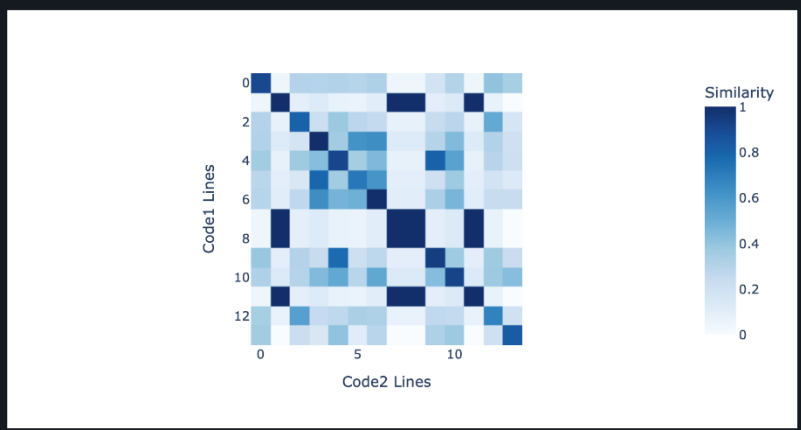
Copied Lines:

- Line 4
- Line 7

Suggestions:

- Convert for loop to while loop
- Rename variables
- Change function names
- Break code into smaller functions
- Modify logic structure

🔥 Similarity Heatmap



Code Comparison

Code 1

```
# Program to print numbers and calculate sum

def calculate_sum(n):
    total = 0
    for i in range(n):
        total += i
    return total


for i in range(5):
    print(i)

result = calculate_sum(5)
print("Sum is:", result)
```

Code 2

```
# Program to print numbers and compute sum

def compute_sum(n):
    total = 0
    for num in range(n):
        total = total + num
    return total


for x in range(5):
    print(x)

output = compute_sum(5)
print("Sum is:", output)
```

✨ Plagiarism-Free Code (Auto Modified)

```
# Program4 to15 print71 numbers46 and75 calculate11 sum33

def calculate_sum(n):
total52 = 0
for i37 in range(n):
total52 += i37
return total52


for i37 in range(5):
print(i)

result91 = calculate_sum(5)
print("Sum is:", result)
```

Previous Results

ID	Language	Score	Result
2	python	83.07	⚠️ Possible Plagiarism Detected
1	python	83.07	⚠️ Possible Plagiarism Detected